NATIONAL UNIVERSITY OF SINGAPORE

B

CS1101S — PROGRAMMING METHODOLOGY

AY2023/2024 SEMESTER 1

READING ASSESSMENT 1

Time allowed: **40 minutes**

QUESTION PAPER

INSTRUCTIONS

1. This **Question Paper** contains **19 Questions**, and comprises **10 printed pages**, including this page.
2. You are also provided with an **Answer Sheet** to write your answers. It comprises **2 printed pages**.
3. Use a **2B pencil** to **write** your **Student Number** in the designated space on the front page of the **Answer Sheet**, and **shade** the corresponding circle **completely** in the grid for each digit or letter. DO NOT WRITE YOUR NAME.
4. You must **submit only** the **Answer Sheet** and no other documents.
5. All questions must be answered in the space provided in the **Answer Sheet**; no extra sheets will be accepted as answers.
6. For **multiple choice questions (MCQ)**, use a **2B pencil** to **shade** in the **circle** of the correct answer **completely**.
7. Each question has one correct answer. **1 mark** is awarded for each correct answer and there is no penalty for a wrong answer.
8. The **full score** of this assessment is **19 marks**.
9. Answer **all questions**.
10. This is a **closed-book** assessment.

(1) What is the **alphabet letter** that is in the box at the **top-right corner** on the **front page** of this **question paper**? (**Important**: Please make sure your answer is correct because it determines how your answers to all the subsequent questions will be graded.)

A. A
B. B
C. C
D. D

Section A

(2) What is the result of evaluating the following Source §1 program?

```
const x = 1;

function fun(x, y) {
    const z = 2;
    return (y, z) => 100 * x + 10 * y + z;
}

fun(4, 5)(6, 7);
```

A. `147`
B. `467`
C. `456`
D. `156`
E. Error: one or more names is/are not declared before being used
F. Error: one or more names is/are redeclared

(3) What is the result of evaluating the following Source §1 program?

```
const x = f => f;
x(1);
```

A. No result: the evaluation leads to an error "`f` is not a function"
B. No result: the evaluation leads to an error "`x` is not a function"
C. A function that always returns `1`
D. A function that always returns a function
E. No result: the evaluation leads to an error "`1` is not a function"
F. A function that always returns its argument
G. `1`

(4) Consider the following Source §1 program.

```
function f(x) {           // A
    return x;
}
function g(y) {
    const f = z => z + 1;
    return y + f(y);      // B
}
function h(w) {
    return (f => f)(w);   // C
}
h(g(f(1)));               // D
```

Which occurrence(s) of the name `f` is/are in the scope of the declaration in line ***A***?

A. No occurrences
B. The occurrences in lines ***C*** and ***D***, only
C. The occurrences in lines ***B*** and ***D***, only
D. The occurrences in lines ***B*** and ***C***, only
E. The occurrence in line ***C***, only
F. The occurrence in line ***D***, only
G. The occurrences in lines ***B***, ***C***, and ***D***
H. The occurrence in line ***B***, only

(5) What is the result of evaluating the following Source §1 program?

```
const f = (x => x)(y => 2);
f(1);
```

A. `2`
B. No result: the evaluation leads to an error "`2` is not a function"
C. No result: the evaluation leads to an error "`f` is not a function"
D. A function that always returns `1`
E. `1`
F. A function that always returns its argument
G. A function that always returns a function
H. A function that always returns `2`

(6) What is the result of evaluating the following Source §1 program?

```
const f = x => x;
const g = y => g;
const h = z => h(z);
f(g(h))(1)(2);
```

A. No result: the evaluation leads to an error "`2` is not a function"
B. `1`
C. `2`
D. A function that always returns its argument
E. A function that always returns `1`
F. No result: the evaluation does not terminate
G. A function that always returns a function
H. A function that always returns `2`

Section B

We know that in **Source §1**,

(1) the left operand of a binary operation is evaluated before the right operand, and

(2) when there are multiple arguments in a function application, the argument expressions are evaluated from left to right, i.e. the leftmost argument expression is evaluated first, and the rightmost last.

In some of the following questions, the pre-declared `display` function is used in the programs. The `display` function displays/prints the value of its input argument in the REPL, *and* returns the value of its input argument. For example, `display(2 * 5)` prints `10` and returns `10`; and `display(1 > 2)` prints `false` and returns `false`.

(7) What is the sequence of values printed by the `display` function when the following Source §1 program is evaluated?

```
const D = display;

function fun(x, y) {
    return D( D(6 + x) * D(5 + y) ); // (6 + x) * (5 + y)
}
fun( D(4), D(3) ); // equivalent to fun(4, 3);
```

A. `4 3 6 10 5 8 80`
B. None of the other options is the correct answer
C. `4 3 10 8 80`
D. `4 3 80 6 10 5 8`
E. `4 3 80 10 8`
F. `4 3 6 4 10 5 3 8 80`
G. `4 3 80 10 6 4 8 5 3`

(8) What is the sequence of values printed by the `display` function when the following Source §1 program is evaluated?

```
function fun(n) {
    if (n <= 1) {
        return 1;
    } else {
        display(n);
        return 2 * fun(n - 1);
    }
}

fun(6);
```

A. None of the other options is the correct answer
B. `32  16  8  4  2`
C. `6  10  8  6  4  2`
D. `2  4  8  16  32`
E. `6  5  4  3  2`
F. `6  12  24  48  96`
G. `2  3  4  5  6`

(9) What is the sequence of values printed by the `display` function when the following Source §1 program is evaluated?

```
function fun(n) {
    if (n <= 1) {
        return 1;
    } else {
        const x = 2 * fun(n - 1);
        display(x);
        return x;
    }
}

fun(6);
```

A. `2  4  6  8  10  12`
B. `2  4  8  16  32`
C. `6  5  4  3  2`
D. None of the other options is the correct answer
E. `2  3  4  5  6`
F. `12  10  8  6  4`
G. `32  16  8  4  2`

(10) What is the sequence of values printed by the `display` function when the following Source §1 program is evaluated?

```
function fun(d, n) {
    display(d);
    return n <= 1 ? 1 : fun(d + 1, n - 1) + fun(d + 1, n - 2);
}

fun(0, 4);
```

A. `0 1 2 3 2 3 1 2 0 1 2 1 2`
B. `0 1 2 2 1 2 2 3 3`
C. `0 1 1 2 2 3 3 2 2`
D. `0 1 2 3 3 2 1 2 2`
E. `0 1 1 2 2 2 2 3 3`
F. `0 1 2 3 4 5 6 7 8`
G. None of the other options is the correct answer

(11) What is the sequence of values printed by the `display` function when the following Source §1 program is evaluated?

```
function D(m, x) {
    display(m);
    return x;
}

function fun(g) {
    D("fun", "fun");
    return D("g()", g());
}

fun( D( "arg", () => D( "4", D("3", 3) + D("1", 1) ) ) );
// equivalent to fun( () => 3 + 1 );
```

A. `"arg" "fun" "3" "1" "4" "g()"`
B. `"fun" "g()" "3" "1" "4" "arg"`
C. `"arg" "4" "3" "1" "fun" "g()"`
D. None of the other options is the correct answer
E. `"fun" "g()" "arg" "4" "3" "1"`
F. `"arg" "4" "fun" "3" "1" "g()"`
G. `"3" "1" "4" "arg" "fun" "g()"`

Section C

(12) What kind of process does the following function give rise to for any *integer* arguments `x` > 0 and `y` > 0?

```
function foo(x, y) {
    return x === 0
           ? y === 0
           ? 1
           : foo(x, y - 1)
           : foo(x - 1, y);
}
```

A. A process that is neither iterative nor recursive
B. A substitution process
C. A recursive process
D. An infinite process
E. An iterative process

(13) What kind of process does the following function give rise to for any *integer* argument `n` > 1?

```
function foo(n) {
    return n === 1
           ? 2
           : foo(n - 1) + foo(n - 1);
}
```

A. An infinite process
B. A process that is neither iterative nor recursive
C. A recursive process
D. A substitution process
E. An iterative process

(14) What kind of process does the following function give rise to for any *integer* argument `n` > 0?

```
function foo(n) {
    const g = x => x - 1;
    return n === 0 ? 0 : foo(g(n));
}
```

A. An iterative process
B. A recursive process
C. An infinite process
D. A process that is neither iterative nor recursive
E. A substitution process

(15) What kind of process does the function `goo` give rise to for any *number* argument `x` > 1, and any unary function `fun`? Note that `fun` always gives rise to an iterative process for any number argument, and it always terminates.

```
function goo(fun, x) {
    return x <= 1
           ? fun(x)
           : fun(goo(fun, x / 2));
}
```

A. An infinite process
B. A process that is neither iterative nor recursive
C. A substitution process
D. An iterative process
E. A recursive process

Section D

(16) We specify that the function `AA`, when applied to any three number arguments, should return the smallest value of the three.

```
function AA(x, y, z) {
    return x > y
           ? y < z
           ? y
           : z
           : x;
}
```

Which one of the following statements is correct?

A. The function `AA` does not meet the specification because it can be applied to *non-positive* and/or *non-integer* argument values.
B. The function `AA` does not meet the specification because it does not check whether the arguments are valid inputs.
C. The function `AA` does not meet the specification because its returned value is incorrect for some valid argument values.
D. The function `AA` does not meet the specification because it is an inefficient way to compute the result.
E. The function `AA` meets the specification.

(17) We specify that the function `CC`, when applied to any two number arguments, should return the sum of the squares of the two input values. For example, `CC(3, 4)` should return the value `3*3 + 4*4 = 25`.

```
function acc(op) {
    return (x, y) => op(op(y) + op(x));
}

const CC = acc(x => x * x);
```

Which one of the following statements is correct?

A. The function `CC` meets the specification.
B. The function `CC` does not meet the specification because it has syntax error(s).
C. The function `CC` does not meet the specification because it does not return the correct results for ***all*** valid argument values.
D. The function `CC` does not meet the specification because it does not return the correct results for ***some but not all*** valid argument values.
E. The function `CC` does not meet the specification because it causes a run-time error.

(18) We specify that the function `DD`, when applied to any two *positive integers*, `n` and `p`, returns the value of the `p`th digit from the right in the decimal representation of `n`. If `n` has fewer than `p` digits, `DD` returns `-1`. For example, `DD(98765, 4)` should return `8`, and `DD(321, 4)` should return `-1`.

```
function DD(n, p) {
    return math_pow(10, p - 1) > n
           ? -1
           : math_floor(n / math_pow(10, p - 1)) % 10;
}
```

- `math_pow(base, exp)` — returns the result of raising `base` to the power of `exp`.
- `math_floor(x)` — returns the greatest integer that is equal to or less than `x`.

Which one of the following statements is correct?

A. The function `DD` does not meet the specification because it does not check if `n` > `p`.

B. The function `DD` meets the specification.

C. The function `DD` does not meet the specification because it does not return the correct results for some valid argument value(s).

D. The function `DD` does not meet the specification because it has syntax error(s).

E. The function `DD` does not meet the specification because it can be applied to *non-positive* argument values.

(19) We specify that the function `FF`, when applied to any *non-negative integer* argument `n`, returns `n`!, the factorial of `n`.

```
function FF(n) {
    function helper(t, s) {
        return t === 0 ? s : helper(t - 1, s + n);
    }
    return n === 0 ? 1 : helper(FF(n - 1), 0);
}
```

Which one of the following statements is correct?

A. The function `FF` meets the specification.

B. The function `FF` does not meet the specification because it runs into infinite recursion for some valid argument value(s).

C. The function `FF` does not meet the specification because it is an inefficient way to compute the factorial.

D. The function `FF` does not meet the specification because it cannot be applied to *negative integer* argument values.

E. The function `FF` does not meet the specification because it does not produce the correct results for some valid argument value(s).

——— END OF QUESTIONS ———